

# PIXEL FÄRBEN

## Ausgabe

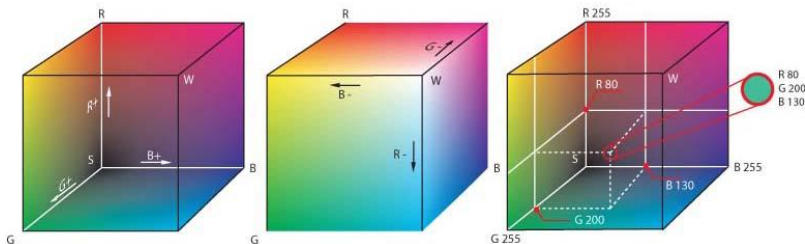
- Grafik kann auf vielen Geräten ausgegeben werden
  - Bildschirm
  - Beamer
  - (3D-)Drucker
  - Plotter
  - Laser
- Wir konzentrieren uns auf Bildschirmausgabe

# Bildschirm

- Zweidimensionale Punktmatrix (*pixel*)
  - Pixel hat x-/y-Koordinate
  - Üblicherweise ist links oben 0,0
  - Achsen positiv
- Auflösung = Anzahl Pixel pro Zeile mal Anzahl Pixel pro Spalte
  - z.B. 640x480, 1920x1080
- Seitenverhältnis (*aspect ratio*)
  - z.B. 4:3, 16:9, 16:10

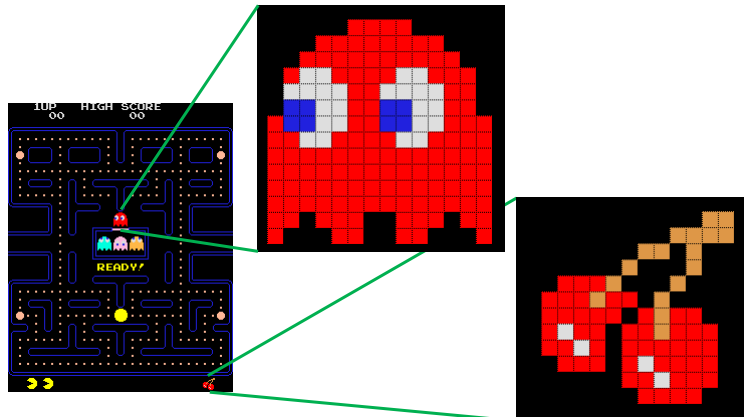
# Farbe

- Typisch: RGB für Rot, Grün, Blau
  - Oft Werte zwischen 0-1 oder 0-255 (byte)
- Farbe = Punkt im Farbwürfel mit Koordinaten R,G,B



# Bitmap-/Pixelgrafik

- Kopieren vorgefertigter Pixel in die Bildschirmmatrix



Computergrafik - Prof. Dr. Tobias Lenz

26

Bilder: Wikipedia

## Soweit, so GdM

- Jeder hat gelernt, Pixelbilder zu “bauen”

for  $y=0 \dots \text{Höhe} - 1$

for  $x=0 \dots \text{Breite} - 1$

pixel(x,y) = RGB-Wert

- In CG geht das ähnlich und doch anders

Computergrafik - Prof. Dr. Tobias Lenz

27

## Rechenbeispiel Farbzweisungen

- 4k-Auflösung:  $3840 \times 2160 = 8.294.000$
- Bildwiederholfrequenz 60Hz: ca. 500Mio/s
- Annahme: Prozessor mit 2,5 GHz (2 Kerne), 1 Befehl pro Takt
  - $2 \times 2,5 \text{ Mrd} / 500 \text{ Mio} = 10$  Befehle pro Pixel
- Wie soll mit unter 10 Assemblerbefehlen pro Pixel ein realistisches Bild entstehen?
- Dabei ist CPU zu 100% ausgelastet!!

## Spezielle Grafikhardware

- Heutige Grafikkarten haben über 10.000 Kerne in Spezialprozessoren (GPU)
- Diese Rechenkerne arbeiten **parallel**
  - Sehr schnell, wenn die Berechnungen unabhängig sind (tausendfach schneller!)
  - Andere Denkweise beim Programmieren erforderlich
- Prozessor (CPU) fast gar nicht belastet und parallel nutzbar für andere Aufgaben

## Doch anders als GdM

- Pixel werden parallel eingefärbt
- Reihenfolge nicht bekannt
- Kein Zugriff auf Nachbarn
  - Vielleicht noch gar nicht fertig? Unabhängig!!
- Wir implementieren Methode:

```
sagMirFarbe(x,y) {      ← Methode wird irgendwann aufgerufen
    return “unser gewünschter RGB-Wert”;
}
```

## Eigener Prozessor=eigene Befehle

- Spezielle Hardware für Grafik optimiert
- Diverse Programmiersprachen
  - Cg (wird nicht mehr weitergeführt)
  - HLSL für DirectX
  - GLSL für OpenGL
- Alle an C angelehnt, also ähnlich zu Java
- Wir benutzen OpenGL, also auch GLSL (*OpenGL Shading Language*)
- Programme heißen “Shader”

# Wie sieht ein GLSL-Shader aus?

- Eigene Programmiersprache mit
  - Datentypen für Variablen
  - Typische Kontrollstrukturen
  - Eingebauten Funktionen
  - Selbst definierten Funktionen
  - Einigen „magischen Konstanten“
- Keine Objektorientierung!

## Datentypen

- Wie in Java: Datentypen für Bits, ganze Zahlen, Fließkommazahlen
  - `bool`, `int`, `float`, `double`
- Anders als Java: Ganze Zahlen ohne Vorzeichen (Werte bis  $2^{32}-1$ )
  - `uint`
- Im normalen Alltag kommt man mit `int` und `float` sehr weit



# Casting

- Typenumwandlung in Java

```
float f = 5.8f;
int i = (int) f;
```
- Typenumwandlung in GLSL ist einfach nur “anders geklammert”

```
float f = 5.8; //nicht 5.8f
int i = int(f);
```

# Array-Datentypen

- (Fast) wie in Java
  - Größe beim Anlegen festlegen
  - .length() gibt Länge zurück
  - Erst ab OpenGL 4.3 auch mehrdimensional
- Beispiele

```
float a[3] = float[3](1.0, 0.5, 3.9);
a[2] = 0.7;
int len = a.length(); //ergibt 3

double b[10][3]; //erst ab OpenGL 4.3
```

# Vektor-Datentypen

- Vektoren zu jedem Typ mit 2, 3 oder 4 Elementen
  - `bvec2` – bool-Vektor mit 2 Elementen
  - `ivec3` – int-Vektor mit 3 Elementen
  - `uvec4` – uint-Vektor mit 4 Elementen
  - `vec2` – float-Vektor mit 2 Elementen
  - `dvec3` – double-Vektor...
- Am meisten verwendet: `vec2`, `vec3`, `vec4` 

# Konstruktor

- Wie in Java, nur ohne “new”
- `ivec2 iv = ivec2(1, -3);`
- `vec3 v = vec3(1.0, -3.217, 0.3);` 
- `vec4 v4 = vec4(v, 0.0);`
  - 4-elementiger Vektor ist 3-elementiger Vektor und eine zusätzliche Zahl
- `vec2 a = vec2(1.0, 2.0);`
- `vec2 b = a; //Ist Kopie! Keine Referenz!`

# Swizzles

- Zugriff auf einzelne Komponenten eines Vektors mit `.`
- `vec3` `v`;
- `float` `x` = `v.x`; //gibt x-Koordinate 
- `vec2` `p` = `v.xy`; //gibt 2d-Vektor mit x und y
- `vec2` `q` = `v.yx`; //gibt 2d-Vektor mit y und x
- `vec4` `w` = `v.xyyx`; //ist erlaubt...
- `vec4` `farbe`;
- `farbe.rgba` == `farbe.xyzw`

# Matrix-Datentypen

- Matrizen der Größe  $n \times m$ , mit  $n$  und  $m$  zwischen 2 und 4
  - `mat2x3`, `mat4x3`, ...
- Wichtiger Sonderfall: Quadratische Matrizen
  - `mat2`, `mat3`, `mat4`
- Auch mit double-Werten
  - `dmat3`, `dmat2x4`, ...
- Am meisten verwendet: `mat3`, `mat4` 

# Matrix-Zugriff

- Zugriff auf eine Matrix wie auf ein zweidimensionales Array von floats  
`mat3 m;`  
`float f = m[2][1];`
- Oder als Array von Spaltenvektoren  
`mat3 m;`  
`vec2 v = m[0].xy;`  
`m[1].zxy = vec3(1.0, 2.0, 3.0); //ergibt 3,1,2`

# Modifizier

- In Java z.B. static oder final
- In GLSL gibt es Konstanten (nie änderbar)  
`const int i = 5;`  
`i = 3; //Compiler-FEHLER!`
- Globale Werte
  - „Einheitlich“ während eines Zeichendurchlaufs
  - Später sehr wichtig, jetzt noch nicht`uniform int i = 5;`



# Strukturen

- Wie in C/C++ ;-)
- Ähnlich zu Java-Klassen
- Verbindet mehrere Datentypen zu einem komplexeren Typ
- Keine Methoden – keine Objektorientierung!
- Zugriff auf Elemente mit .

## Strukturen: Beispiel

```
struct meinTyp {  
    int x;  
    float y;  
}  
hierKnnDirkt1VriableDeklariertWerdn;  
  
hierKnnDirkt1VriableDeklariertWerdn.x = 2;  
  
struct meinTyp a = meinTyp(0, 1.0);
```

## Reservierte Variablen

- Einige reservierte Namen mit spezieller Bedeutung
- Fangen immer mit `gl_...` an
- Existieren einfach, ohne Deklaration o.ä.
- Beispiel: Wenn wir Pixel einfärben wollen, dann können wir die Pixel-Koordinaten abfragen über `gl_FragCoord` (Typ ist `vec4`)  
`vec2 pixelXY = gl_FragCoord.xy` 

## Typische Konstrukte

- `if ( ... ) { ... } else { ... }` 
- Schleifen
  - `for ( ... ) { ... }`
  - `while ( ... ) { ... }`
  - `do { ... } while ( ... )`
- Funktionieren wie in Java

```
if (a > 5 && b <= 7) {  
    float lokaleVariable = 3.1;  
}
```

# Funktionen

- Viele mathematische Funktionen

- min, max
- cos, sin, tan, acos, asin, atan
- ceil, floor, round
- exp, pow, sqrt

```
float f = sqrt(cos(0.1));
```

```
float m = max(1.0, f);
```

```
int im = max(5, 7);
```

```
int error = max(5, 7.0); //?! Auto-cast?
```

## Noch Mehr Funktionen

- Wichtige Vektor/Matrix-Operationen
  - Skalarprodukt (dot), Kreuzprodukt (cross)
  - normalize, reflect, transpose
- Besprechen wir später genauer, wenn wir den mathematischen Teil betrachten
- Übrigens: KEINE REKURSION!

# Eigene Funktionen

- Programm beginnt bei *main* (ohne Parameter)
- Eigene Funktionen wie Java-Methoden
  - Rückgabetyp Funktionsname(Parameter)
- Anders als Java: Funktion muss deklariert sein, bevor sie aufgerufen werden kann
- Lokale Variablen innerhalb einer Funktion deklariert, globale außerhalb

## Eigene Funktionen: Beispiel

```
const int globaleVariable = 2;
int gibMir5() { return 5; } //muss vor beispiel() stehen

void beispiel(in int a, out int b, inout int c) {
    a = gibMir5();
    b = a;
    c += globaleVariable;
}

void main() {                                //hier startet das Programm
    int x = 3, y, z = 4; //lokale Variablen
    beispiel(x, y, z);    //welche Werte haben x,y,z?
}
```

# Alle Funktionen

- Offizielle Spezifikation (>1000 Seiten PDF)
  - [https://www.khronos.org/registry/OpenGL/index\\_gl.php](https://www.khronos.org/registry/OpenGL/index_gl.php)
- Offizielle Webseite zum Suchen
  - <https://www.khronos.org/registry/OpenGL-Refpages/gl4/>
- Offizielles Wiki
  - <https://www.khronos.org/opengl/wiki/>
- Inoffizielle, beliebte Webseite zum Suchen
  - <http://docs.gl/>

## Wollten wir nicht Pixel färben?

```
#version 330
//globale Ausgabe unseres Programms (out)
//Typ ist vec3 für RGB (jeweils 0.0 bis 1.0)
out vec3 pixelFarbe;    //Name beliebig

void main() {
    pixelFarbe = vec3(1.0, 0.0, 0.0);
}
```